Batch Whitening for Vision and LLMs: Bridging Theory and Implementation

Anonymous Authors[1]

Abstract

[TODO] 1. Experiments for vision + comparison to cho2021improving
2. LLM experiments. GPT2 is preferred over nanoGPT
3. Ablation study + observing the condition number w/ and w/o BW. 3. Complete missing sections

Batch Whitening (BW) improves optimization by decorrelating feature channels, but its adoption in large-scale models, particularly LLMs, has been limited by statistical, structural, and causal challenges. In this work, we introduce a stable and scalable BW framework for both vision and language models. For vision, we propose three complementary techniques – Reorder & Split, Controlling Estimation Error, and Handling Correlated Channels – that improve whitening performance and reduce estimation noise. Building on these insights, we develop Causal Batch Whitening (CBW) for autoregressive LLMs, preserving causality while maintaining computational efficiency and reliable covariance estimation. Our approach integrates naturally with modern deep learning architectures, improving optimization stability, gradient isotropy, and downstream performance. Extensive empirical evaluation demonstrates that BW and CBW achieve competitive or superior results compared to LayerNorm, GroupNorm, and prior whitening methods.

1. Introduction

Normalization layers are fundamental components in modern deep learning. Methods such as Batch Normalization, Layer Normalization, and Group Normalization improve optimization stability and generalization by regulating feature statistics during training. A more principled alternative is *whitening*, which removes correlations across activation channels by transforming the input $\mathbf{x}$ of a layer so that its covariance satisfies $\Sigma_x \approx I$. Whitening is theoretically appealing: in linear models, the Hessian is exactly Σ_x (Amari, 1998; Martens, 2010), and in deep nonlinear networks, dominant curvature terms such as the Gauss–Newton or Fisher matrices inherit the anisotropy of Σ_x (Desjardins et al., 2015; Cho et al., 2021). Empirically, whitening improves gradient isotropy, reduces co-adaptation, and stabilizes training across a range of architectures.

Despite these benefits, *practical* batch whitening (BW) has seen limited adoption. Existing whitening techniques perform reliably only in restricted settings – primarily CNNs on vision tasks – and often suffer from instability, reduced accuracy, or high computational cost. Three challenges have prevented whitening from becoming a widely used normalization mechanism:

1. **High-dimensional covariance estimation.** Modern architectures involve hundreds or thousands of channels, making covariance estimation and inversion noisy and computationally demanding.
2. **Strong inter-channel correlations.** Real activation patterns exhibit structured dependencies. Prior heuristics such as simple channel grouping or shuffling fail to capture these correlations and lead to suboptimal whitening.
3. **Causality constraints in sequence models.** For autoregressive tasks, whitening cannot rely on full-sequence statistics without violating causality. This has made whitening incompatible with large language models (LLMs), where LayerNorm remains the default.

As a result, whitening has never been deployed in LLMs and has shown only partial success in vision models. In this work, we revisit batch whitening from first principles and develop a framework that addresses all three challenges, enabling whitening to operate effectively across both *vision* and *language* domains.

Our Contributions

We introduce the first practical and scalable Batch Whitening framework applicable to both vision architectures and

[1]Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

large language models. Our contributions are:

- **A stable and accurate BW method for vision models.** We introduce three techniques that collectively address long-standing barriers in whitening: (i) *Reorder & Split*, a structural reorganization that places whitening immediately before the linear operation and moves the nonlinearity after it, preserving the orthonormal feature space; (ii) *Controlled Estimation Error*, which stabilizes covariance estimation through temporal averaging and adaptive block sizes chosen to keep estimation noise low; and (iii) *Handling Correlated Channels*, a shrinkage-based regularization that injects small independent noise to reduce harmful correlations and ensure the covariance is well-conditioned. Together, these significantly improve whitening performance over prior methods in CNNs.
- **Causal Batch Whitening (CBW): the first whitening method compatible with LLMs.** We propose a novel *causality-preserving covariance estimator* that maintains autoregressive correctness while enabling effective whitening during both training and inference. No prior whitening method supports causal sequence modeling.
- **A matrix-friendly, parallelizable implementation.** We design the CBW computation to be fully expressed as matrix operations, avoiding explicit loops. This allows efficient, parallel execution on GPUs, making it practical for large-scale modern LLMs.

By addressing the statistical, structural, and causal challenges that have limited whitening in practice, this work establishes Batch Whitening and Causal Batch Whitening as viable normalization mechanisms for large-scale deep learning. An empirical evaluation across both vision and language tasks demonstrates that our whitening layers reduce gradient anisotropy, stabilize long-horizon optimization, and achieve competitive or improved accuracy compared to LayerNorm, GroupNorm, and existing whitening variants.

2. Related Work

[TODO] Add

3. BW for Convolutional Neural Networks

BW improves optimization dynamics in convolutional neural networks by decorrelating feature channels and equalizing their scales. In practice, implementing BW requires addressing several challenges, including maintaining the orthonormality of features through nonlinearities, controlling estimation error in high-dimensional settings, and handling strongly correlated channels. In the following subsections, we present three key techniques that collectively form a robust and practical whitening method. We then discuss how these components are combined into a unified algorithm, including temporal updates across iterations, to yield the final Practical Batch Whitening (PBW) approach.

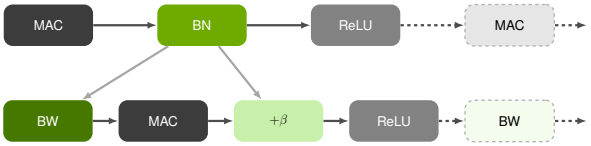


Figure 1. [TODO] (1) Add caption, (2) why the $+\beta$ is after the MAC and not part of it?

3.1. Reorder and Split

[TODO] Review details

In Batch Norm (BN) it is standard to place a nonlinearity such as ReLU immediately after normalization, since BN only rescales channels and does not enforce any structural constraint that must be preserved. In contrast, Batch Whitening produces an orthonormal feature space that should remain intact until the subsequent linear (MAC) operation. Inserting a nonlinearity between whitening and the next MAC destroys this orthonormality, undermining the purpose of whitening. Therefore, for BW we propose to reorder and split the block: apply whitening immediately before the linear transformation, and place the nonlinearity after that linear step, ensuring that the decorrelated representation is consumed unaltered by the MAC. Figure 1 illustrates this proposed ordering. [TODO] Explain the figure. Why the $+\beta$ is after the MAC and not part of it?

[TODO] Need to add a comment here that some CNNs, like EfficientNet, are already designed like this and require no adaptations. For the rest of the CNNs, an adaptation of the net is needed (it is not simply replacing the BN with BW).

3.2. Controlling Estimation Error

[TODO] Review details

In BW, the covariance matrix is estimated from a set of vectors (e.g., token or feature representations). The accuracy of this estimate improves with the number of vectors V used, and degrades as the number of channels C increases. Intuitively, the estimation error grows with C because there are more covariance entries to estimate, and each entry has variance proportional to $1/V$ (Ledoit & Wolf, 2004; Huang et al., 2018). Formally, the expected Frobenius-norm error of the covariance estimate can be approximated as:

$$e \sim \sqrt{\frac{C^2}{2V}}. \tag{1}$$

Thus, when C is large and V is fixed, the estimate can be highly noisy. Since the activations entering a whitening layer are normalized, the entries of their covariance lie roughly in $[-1, 1]$. An error of comparable magnitude would therefore swamp the very correlations that whitening is meant to remove, so e must be kept well below 1. Requiring $e \le e_{\text{max}}$ for a target error e_{max} gives the number of samples needed for a reliable estimate:

$$V \ge \frac{C^2}{2e_{\text{max}}^2}. \tag{2}$$

Samples per layer. For a convolutional layer with input of shape $B \times H \times W \times C$, the covariance is taken along the channel axis, and every spatial position of every image is a sample of a C-dimensional vector, so $V = B \cdot H \cdot W$. The batch size B is fixed throughout training, but in a typical CNN the spatial size $H \times W$ shrinks and the channel count C grows with depth. The requirement in (2) grows quadratically in C while the available V shrinks, so no single setting satisfies all layers: early layers have many samples and few channels, while late layers have few samples and many channels.

We control the error with two complementary mechanisms:

1. **Temporal averaging (increasing V).** We maintain a running estimate of the covariance across training iterations (Desjardins et al., 2015):

   $$\mathbf{\Sigma}^{(n)} = (1 - \alpha)\, \mathbf{\Sigma}^{(n-1)} + \alpha\, \mathbf{\Sigma}_{\text{batch}},$$

   where $\alpha \in (0, 1]$ is the momentum, i.e. the weight of the current batch. The running estimate averages over roughly $1/\alpha$ recent batches, so the effective number of samples becomes

   $$V_{\text{eff}} \approx \frac{V}{\alpha}.$$

   A smaller α lowers the noise, but the estimate then lags further behind the changing network, so α should be no smaller than needed.

2. **Block-diagonal covariance (decreasing C).** Instead of estimating a full $C \times C$ covariance, we partition the channels into $M = C/c$ blocks of c channels each and whiten every block independently. This amounts to assuming a block-diagonal covariance: correlations within a block are estimated and removed, while correlations across blocks are ignored. Each estimate now involves only c channels, so the error becomes

   $$e_c \sim \sqrt{\frac{c^2}{2V_{\text{eff}}}}.$$

   The two extremes recover familiar methods: $c = C$ is full whitening, and $c = 1$ estimates only per-channel variances, i.e. Batch Normalization.

Adaptive block size and momentum. Rather than tuning c manually for every layer of every network, we choose it per layer from the error budget. Requiring $e_c \le e_{\text{max}}$ gives the largest block size the available samples can support:

$$c = \min\left(C,\ \left\lfloor e_{\text{max}} \sqrt{2V/\alpha} \right\rfloor\right), \tag{3}$$

which we then round down to a divisor of C so that the blocks partition the channels evenly. This keeps the estimation error roughly constant across layers: early layers, where $V_{\text{eff}} \gg C^2/(2e_{\text{max}}^2)$, get $c = C$ and are fully whitened, while deeper layers with fewer samples and more channels get progressively smaller blocks. If (3) yields $c = 1$, the layer does not have enough samples for any reliable decorrelation, and BW reduces to Batch Normalization.

At the other extreme, when the unconstrained block size exceeds C, the layer has more samples than full whitening requires. The surplus can instead be spent on freshness: we increase α, averaging over fewer batches so the running covariance tracks the network more closely, up to the value at which V/α just meets the requirement:

$$\alpha = \min\left(1,\ \frac{2e_{\text{max}}^2 V}{C^2}\right).$$

In the limit $\alpha = 1$, a single batch already provides enough samples to estimate the full covariance and no temporal averaging is needed. Together, these two rules spend each layer's samples where they are most useful: on larger blocks while full whitening is out of reach, and on faster updates once it is not.

An additional benefit of the block-diagonal formulation is computational efficiency: with $M = C/c$ blocks, covariance computation costs $O(VCc)$ and inversion $O(Cc^2)$, both linear in C for a fixed block size, rather than $O(VC^2)$ and $O(C^3)$ for the full covariance, making BW practical for large-scale models.

3.3. Handling Correlated Channels

[TODO] Review details

Whitening requires computing the inverse square root of the covariance matrix Σ of the input channels. If some channels are strongly correlated, Σ becomes ill-conditioned or even singular, rendering the whitening transform $\Sigma^{-1/2}$ numerically unstable. In extreme cases, near-perfect correlations correspond to very small eigenvalues, which amplify numerical errors when inverted (Desjardins et al., 2015; Huang et al., 2018; Ledoit & Wolf, 2004).

To resolve this, we inject a small independent perturbation into each channel. Let c be a vector representing one channel (or a block of channels), and let $\hat{c}$ be an independent

random vector with the same variance as c. We define a modified channel $\tilde{c} = \sqrt{1-\epsilon}\, c + \sqrt{\epsilon}\, \hat{c}$, where $\epsilon \in (0, 1)$ is chosen close to 0 to effectively reduce correlations without significantly altering the channel variances.

The covariance of $\tilde{c}$ is

$$\begin{aligned}\mathrm{Cov}(\tilde{c}) &= \mathrm{Cov}\big(\sqrt{1-\epsilon}\, c + \sqrt{\epsilon}\, \hat{c}\big) \\ &= (1-\epsilon)\,\mathrm{Cov}(c) + \epsilon\,\mathrm{Cov}(\hat{c}) + 2\sqrt{(1-\epsilon)\epsilon}\,\mathrm{Cov}(c, \hat{c}) \\ &= (1-\epsilon)\,\mathrm{Cov}(c) + \epsilon\,\mathrm{Var}(c)\, I,\end{aligned}$$

where the last equality follows from the independence of c and $\hat{c}$, and the fact that $\hat{c}$ is drawn i.i.d. from the same distribution as c.

In practice, this operation has the effect of shrinking the off-diagonal elements of $\mathrm{Cov}(c)$ by a factor of $1-\epsilon$ while preserving the diagonal variance:

$$\mathrm{Cov}(\tilde{c})_{ij} = \begin{cases} \mathrm{Cov}(c)_{ii} & i = j \\ (1-\epsilon)\,\mathrm{Cov}(c)_{ij} & i \neq j. \end{cases}$$

This simple shrinkage stabilizes the covariance, making it well-conditioned for inversion. It can be viewed as a form of *diagonal loading* or *shrinkage regularization* commonly used in high-dimensional statistics (Ledoit & Wolf, 2004; Chen et al., 2010). By controlling ϵ, we balance between reducing correlations for numerical stability and preserving the original structure of the features. This technique allows reliable whitening even when the original channels are highly correlated, such as in early layers of deep networks or for highly structured feature embeddings in language models.

3.4. Practical Batch Whitening (PBW)

In this section, we combine all the techniques introduced in the previous subsections into a single unified algorithm, Practical Batch Whitening (PBW). PBW integrates the following components: (i) *Reorder and Split*, (ii) *Controlling Estimation Error*, and (iii) *Handling Correlated Channels*.

[TODO] This should be discussed in section "Controlling Estimation Error" where we also should discuss controlling the momentum. After each iteration n, we update the running covariance block estimate for the next step:

$$\boldsymbol{\Sigma}_b^{(n)} = \mathrm{stopgrad}[(1-\alpha)\cdot \boldsymbol{\Sigma}_b^{(n-1)} + \alpha \cdot \boldsymbol{\Sigma}_b],$$

where α is a small momentum parameter (e.g., 0.01). This is analogous to the running mean and variance in batch normalization, but extended to the covariance matrix domain. The key is that we pass $\boldsymbol{\Sigma}_b^{(n)}$ forward as a constant – it stabilizes training but does not influence current gradients.

The final whitening matrix for each block is computed as $W_b = \Sigma_b^{-1/2}$, where $\Sigma^{(-b)}$ denotes the estimated covariance of block b (after applying all the aforementioned techniques). In practice, this matrix inversion can be efficiently implemented using either Newton iterations or Cholesky decomposition.

Algorithm 1 Practical Batch Whitening (PBW)

Require: Input activations $X \in \mathbb{R}^{B\times W\times H\times C}$, previous covariance blocks $\{\boldsymbol{\Sigma}_b^{(n-1)}\}_{b=1}^{M}$, smoothing factor α, noise ϵ
$c = \texttt{adaptive_block_size}(C)$
Flatten spatial dimensions: $\tilde{X} \in \mathbb{R}^{(B\cdot W\cdot H)\times C}$
Number of blocks: $M = C/c$
Split channels to blocks: $\tilde{X} = [X_1, X_2, \ldots, X_M]$
for each block $b = 1, \ldots, M$ **in parallel do**
  Compute block covariance: $\boldsymbol{\Sigma}_b = \texttt{cov}(X_b)$
  Running cov: $\boldsymbol{\Sigma}_b^{\text{updated}} = \alpha\boldsymbol{\Sigma}_b^{(n-1)} + (1-\alpha)\boldsymbol{\Sigma}_b$
  $\boldsymbol{\Sigma}_b^{\text{updated}} = \texttt{decorrelate_channels}(\boldsymbol{\Sigma}_b^{\text{updated}}, \epsilon)$
  Whitening matrix: $W_b = \left(\boldsymbol{\Sigma}_b^{\text{updated}}\right)^{-1/2}$
  Apply whitening: $\tilde{X}_b \leftarrow X_b W_b$
  Temporal update: $\boldsymbol{\Sigma}_b^{(n)} = \texttt{stop_grad}(\boldsymbol{\Sigma}_b^{\text{updated}})$
end for
Concatenate blocks: $\tilde{X} = [\tilde{X}_1, \tilde{X}_2, \ldots, \tilde{X}_M]$
Reshape $\tilde{X}$ back to $B \times W \times H \times C$
return $\tilde{X}$

Algorithm 1 summarizes the complete Practical Batch Whitening (PBW) procedure for training, combining all the techniques described above into a single workflow. During inference, we apply whitening based solely on the accumulated running covariance and disable any updates to the running statistics.

4. Causal BW for Autoregressive Transformers

While in § 3 we presented BW and its implementation details for vision tasks, we now extend the method to autoregressive language models, where causality introduces unique challenges in maintaining valid whitening transformations. In a language model, the token t must not depend on the information from future tokens $t' > t$. Conventional whitening, which computes a covariance matrix across all tokens in a batch, implicitly mixes information from all time steps. This leaks future information into the present and invalidates the autoregressive objective.

To address this, we propose **Causal Batch Whitening (CBW)**, which performs whitening on feature representations while strictly preserving temporal causality. CBW achieves decorrelation of features (improving optimization efficiency) without allowing gradients or normalization statistics from future tokens to influence the current token.

4.1. Mathematical Formulation

[TODO] Consider using C instead of D, to match channel dim in vision section.

Let $\mathbf{x} \in \mathbb{R}^{B\times T\times D}$ denote the activation (input) tensor, where B is the batch size, T is the sequence length, and D is the feature dimension (channels). Each token representation is $\mathbf{x}_{s,t} \in \mathbb{R}^D$.

To preserve causality, the feature-feature covariance for a token t in sequence s must exclude future tokens from that sequence, while still allowing all tokens from other sequences in the batch to contribute. Therefore, the feature-feature covariance for a token t in sequence s is defined as:

$$\boldsymbol{\Sigma}_{s,t} = \frac{1}{(B-1)T}\sum_{s'\neq s}\sum_{i=1}^{T}(\mathbf{x}_{s',i}-\boldsymbol{\mu})(\mathbf{x}_{s',i}-\boldsymbol{\mu})^\top + \frac{1}{t}\sum_{i=1}^{t}(\mathbf{x}_{s,i}-\boldsymbol{\mu})(\mathbf{x}_{s,i}-\boldsymbol{\mu})^\top, \tag{4}$$

where $\boldsymbol{\mu}$ is the mean over the batch and sequence-aware.

This ensures whitening at token t leverages the full batch for statistical efficiency, but strictly prevents future leakage within the sequence.

4.2. Practical Parallel CBW: exclude same-sequence tokens

[TODO] Include Amir's suggestion to use grad/stop-grad so that we can use the full cov and also be memory efficient. Need the algebra to understand how it can be done with mask matrix.

Although Eq. (4) uses as much data as possible without breaking causality, computing the covariance for every token in parallel remains challenging: each token aggregates a different number of valid contributors, and the formulation requires a distinct $B \cdot T$ covariance matrices of size $D \times D$ each, for all the tokens. For realistic LLMs with large T and high feature dimensionality D, this becomes memory- and compute-prohibitive.

A practical compromise – avoiding full per-token CBW while remaining close to it and enabling efficient parallel computation – is to compute, for each sequence s in the minibatch, a *per-sequence* feature covariance that uses *all tokens from other sequences* but *excludes all tokens from sequence s*. Concretely, for a batch $X \in \mathbb{R}^{B\times T\times D}$ we define for each sequence s:

$$\Sigma^{(-s)} = \frac{1}{(B-1)T}\sum_{s'\neq s}\sum_{t=1}^{T}(x_{s',t}-\mu^{(-s)})(x_{s',t}-\mu^{(-s)})^\top,$$

where $\mu^{(-s)}$ is the mean over tokens of all sequences except s. This choice (a) preserves strict causality inside each sequence because statistics never include any token from the same sequence (past or future), (b) yields strong estimates because it aggregates a large number of tokens across other sequences when $B > 1$ and T is large, and (c) requires only B covariance matrices of size $D \times D$, which is computationally manageable and trivially parallelizable on modern accelerators. Note that during the training process, the batch size B is in most cases larger than 1. In the inference stage, we use the existing statistics from the training process, hence indifferent to using batches with a single sequence in the batch.

Algorithm 2 CBW Sequence-Block Covariance

Require: Batch $X \in \mathbb{R}^{B\times T\times D}$, previous block covariance blocks $\{\Sigma_b^{(n-1)}\}_{b=1}^M$, smoothing factor α, block size c
$c = \texttt{adaptive_block_size}(C)$
$M \leftarrow D/c$
Reshape X into blocks: $X \in \mathbb{R}^{B\times T\times M\times c}$
for all $(s,b) \in \{1,\dots,B\}\times\{1,\dots,M\}$ **in parallel do**
  {Collect all tokens from other sequences for block b}
  $x_{s',b} \leftarrow \texttt{concat}\big(X_{s',1:T,b,:}\big)_{s'\neq s}$
  $x_{s',b} \in \mathbb{R}^{(B-1)T\times c}$
  Compute cov on these tokens: $\Sigma_{s,b} \leftarrow \texttt{cov}(x_{s',b})$
  Running cov: $\Sigma_{s,b}^{\text{updated}} = \alpha\,\Sigma_b^{(n-1)} + (1-\alpha)\,\Sigma_{s,b}$
end for
for all $b \in \{1,\dots,M\}$ **in parallel do**
  Temporal update: keep a single running cov per b

$$\Sigma_b^{(n)} \leftarrow \texttt{stop_grad}\left(\frac{1}{B}\sum_{s=1}^{B}\Sigma_{s,b}^{\text{updated}}\right)$$

end for
return $\{\Sigma_{s,b}^{\text{updated}}\}_{s=1..B,\,b=1..M}$

[TODO] The algo is missing grouping and all the features from the previous vision section. Maybe, for simplicity, we can just should how to build the covariance, with groups of channels

Algorithm **??** presents the pseudocode for the parallel CBW procedure. It computes the exclusion covariances $\Sigma^{(-s)}$ for all sequences, obtains their whitening transforms (inverse square roots), and then applies the corresponding transform to each sequence. The entire computation can be vectorized across the batch, thereby removing any loop over B. In practice, the implementation uses efficient matrix operations, ensuring that no Python-level loops over B or T appear in the performance–critical path.

5. Discussion

[TODO] What else? Maybe discuss the option to reduce num of channels while preserving accuracy or convergence rate?

5.1. Why Whitening is Applied Across Features

[TODO] Is this trivial info? Consider removing

Batch Whitening is applied in the *feature* (channel) dimension rather than across tokens (in LLMs) or pixels (in CNNs). In both domains, the model relies on correlations *along the spatial or temporal axis*: CNNs depend on local pixel relationships, and transformers depend on relationships between tokens. Whitening across these axes would explicitly remove the very correlations the architecture is designed to model, thereby degrading attention patterns in LLMs and disrupting spatial structures in vision models.

In contrast, whitening across *feature channels* operates within each token or pixel representation. This preserves structural relationships (token–token or pixel–pixel), while decorrelating hidden channels to improve conditioning, stabilize optimization, and reduce feature redundancy. Thus, feature-domain whitening complements both convolutional and attention-based models: it improves the representation quality without disturbing the core correlation patterns these architectures are meant to capture.

6. Evaluation

[TODO] Add

7. Conclusions

We have presented a practical and scalable BW framework that addresses long-standing statistical and structural challenges in both vision and language models. By extending these techniques to CBW, we enable effective whitening in autoregressive LLMs while preserving causality and computational efficiency. Empirical results demonstrate improved optimization stability, gradient isotropy, and competitive or superior downstream performance, establishing BW and CBW as viable normalization mechanisms for large-scale deep learning.

References

Amari, S.-I. Natural gradient works efficiently in learning. *Neural computation*, 10(2):251–276, 1998.

Chen, Y., Wiesel, A., Eldar, Y. C., and Hero, A. O. Shrinkage algorithms for mmse covariance estimation. *IEEE transactions on signal processing*, 58(10):5016–5029, 2010.

Cho, Y., Cho, H., Kim, Y., and Kim, J. Improving generalization of batch whitening by convolutional unit optimization. In *18th IEEE/CVF International Conference on Computer Vision, ICCV 2021*, pp. 5301–5309. Institute of Electrical and Electronics Engineers Inc., 2021.

Desjardins, G., Simonyan, K., Pascanu, R., and Kavukcuoglu, K. Natural neural networks. In *NeurIPS*, 2015.

Huang, L., Yang, D., Lang, B., and Deng, J. Decorrelated batch normalization. In *CVPR*, 2018.

Ledoit, O. and Wolf, M. A well-conditioned estimator for large-dimensional covariance matrices. *Journal of multivariate analysis*, 88(2):365–411, 2004.

Martens, J. Deep learning via hessian-free optimization. In *ICML*, 2010.

A. Appendix A

TODO